



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

STR

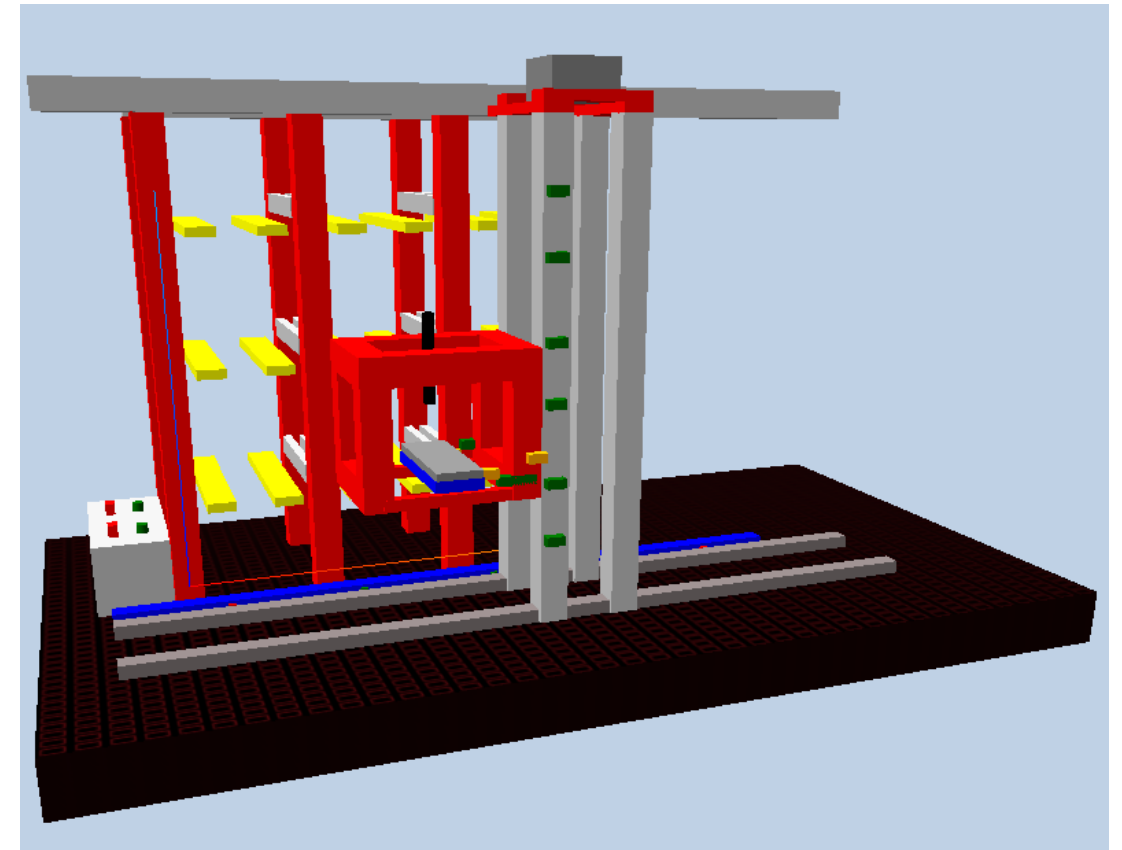
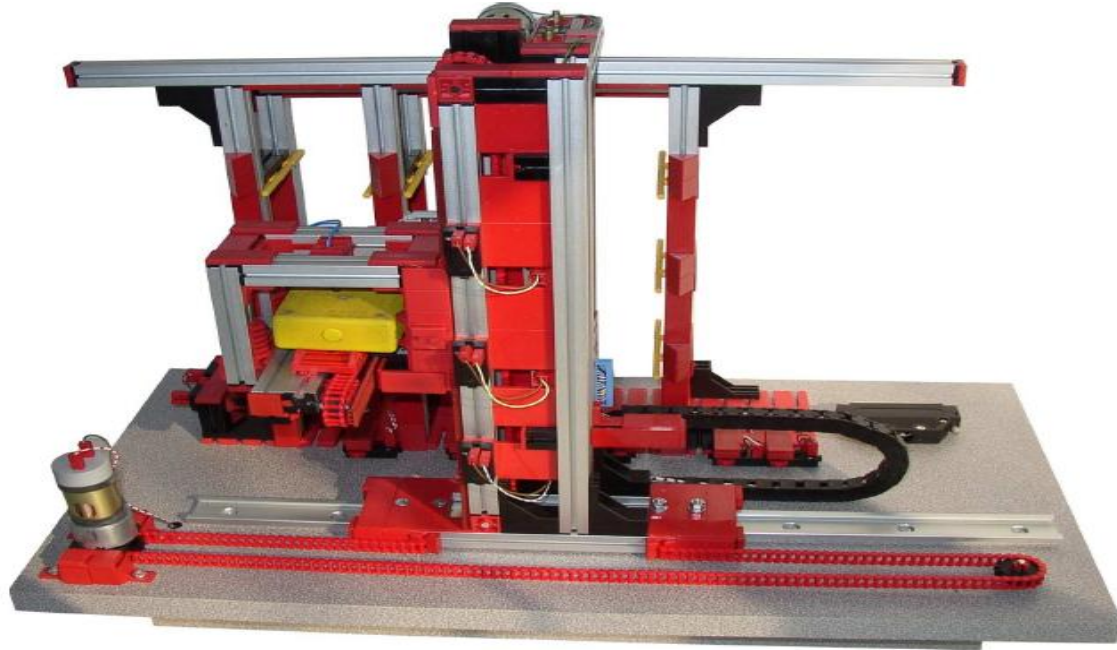
LAB WORK 2: CLASS 2

**JNI Development for Interacting with the
Physical System**

JAVA Real-Time

2025- 2026

LABWORK 2 – COMPACT STORAGE KIT



PART 1: JNI Development

- ☐ [Creation of the JNI interface](#)
- ☐ [Accessing low-level functionality](#)
- ☐ [Testing from Java Application](#)

PART 2: Java Real-Time

- ☐ [The Axis Interface](#)
- ☐ [Developing class Mechanism](#)
- ☐ [Identification of needed Java Threads](#)

PART 1

JNI development for interacting with the physical system



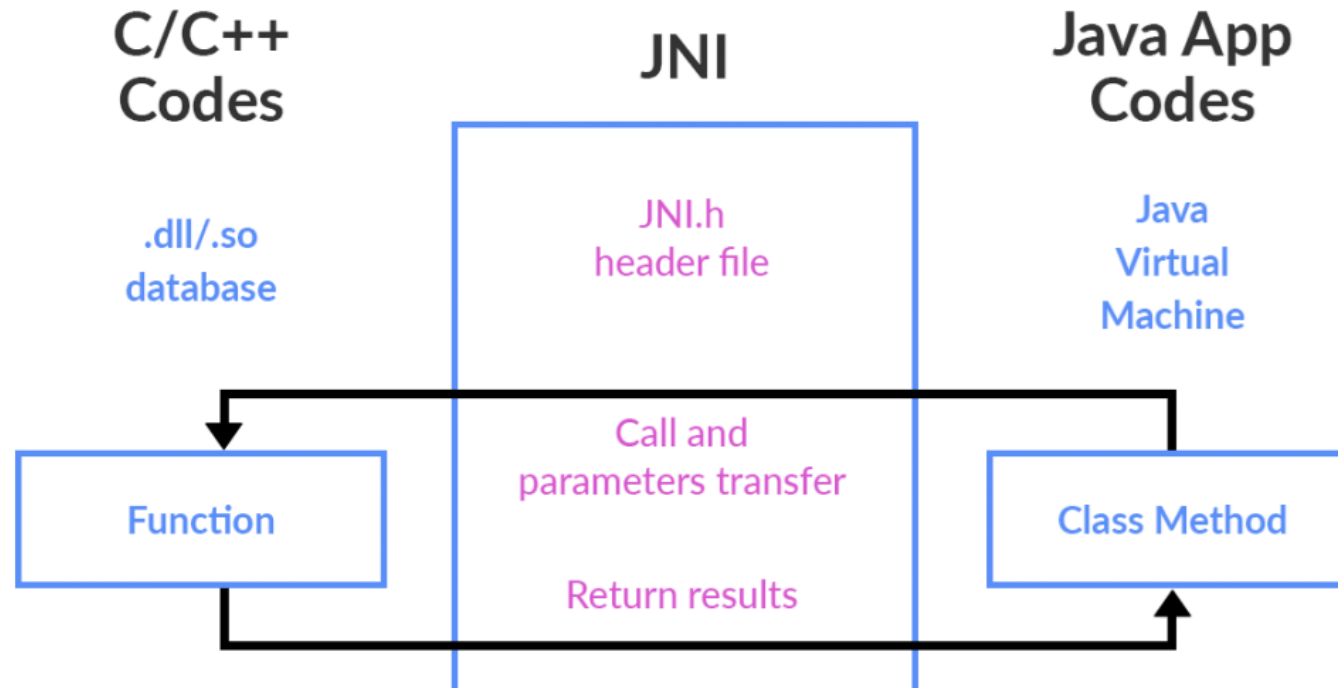
CREATION OF THE JNI INTERFACE



We are now ready to proceed to the next phase, JNI interoperability, for utilization within the Java Application

While you can write applications entirely in Java, there are situations where Java alone does not meet the needs of your application. Programmers use the **JNI** to write Java native methods to handle those situations when an application cannot be written entirely in Java.

<https://docs.oracle.com/javase/7/docs/technotes/guides/jni/spec/intro.html>

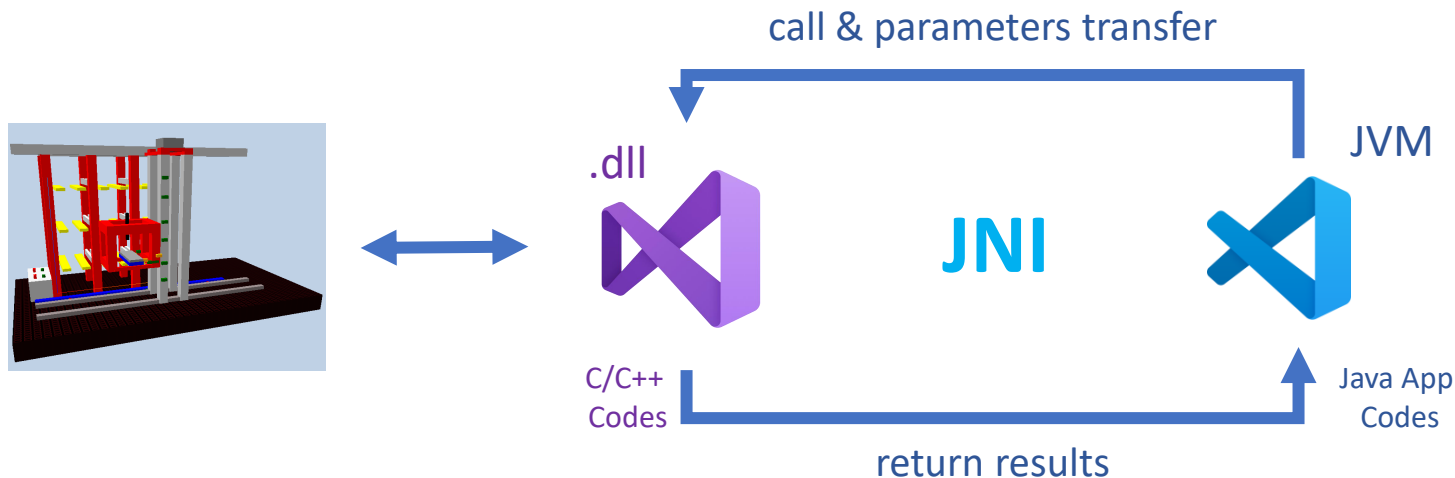
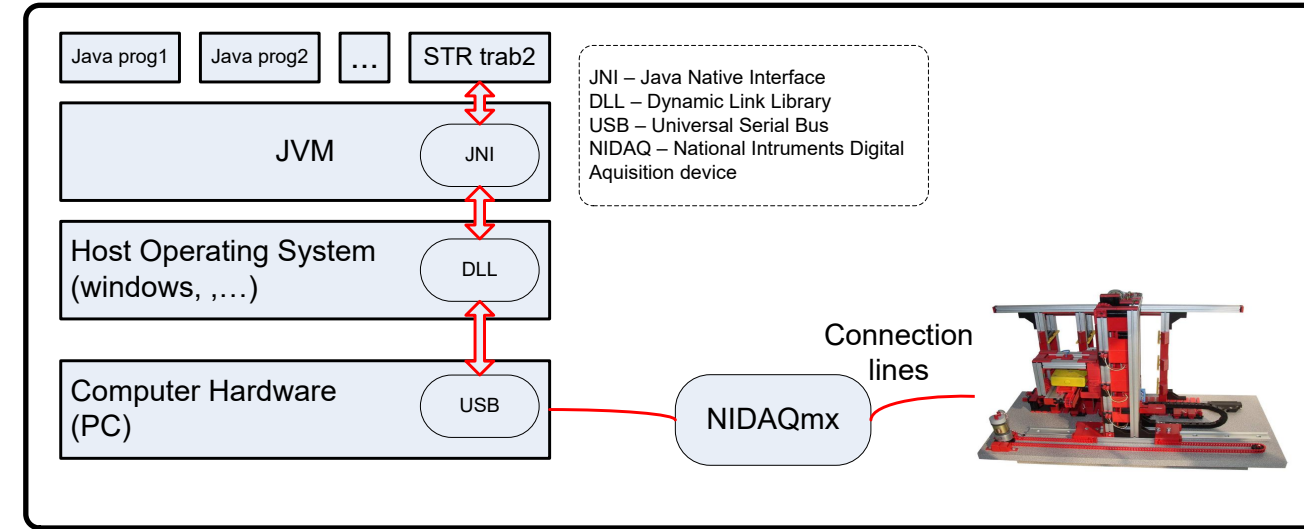


CREATION OF THE JNI INTERFACE

Java Application

- Create a **Java Application** using the **VS Code** environment
- Proceed with the development of the JNI mechanism
- Create a demo **class** to verify that the Java Program can

interact with the Compact Storage



CREATION OF THE JNI INTERFACE



Signatures in Java naming convention (complete table):

Sensors signature functions in C	In Java
int getXPos();	native public int getXPos();
int getZPos();	native public int getZPos();
int getYPos();	native public int getYPos();
(complete the table)	

Actuator signature functions in C	In Java
void initializeHardwarePorts();	native public void initializeHardwarePorts();
void moveXRight();	native public void moveXRight();
void moveXLeft();	native public void void moveXLeft();
void stopX();	native public void stopX ();
(complete the table)	

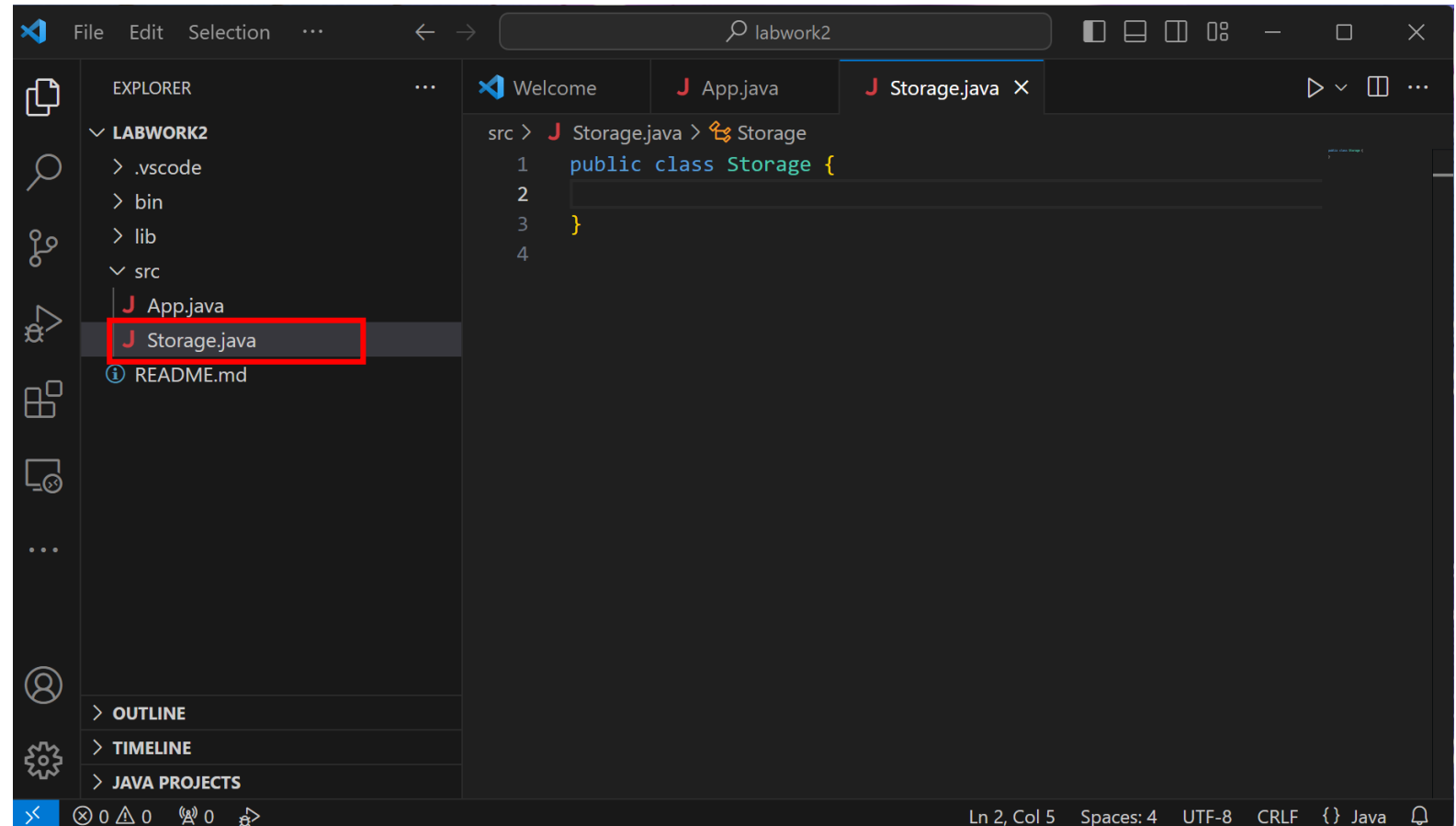


Copy the Java signatures from the tables into a new class **Storage** (see next slide).

CREATION OF THE JNI INTERFACE



- ❑ Open the VS Code **labwork2** project from last week.
- ❑ Create a new java class named **Storage.java**:
 - ❑ Right-click in \src folder, and select New File



CREATION OF THE JNI INTERFACE



- ❑ Copy the signatures from previous table to Storage class.
- ❑ Ensure file is saved.

```
src > J Storage.java > Storage > initializeHardwarePorts()
1  public class Storage {
2
3      static synchronized native public void initializeHardwarePorts();
4      static synchronized native public void moveXRight();
5      static synchronized native public void moveXLeft();
6      static synchronized native public void stopX();
7      static synchronized native public int getXPos();
8
9      static synchronized native public void moveZUp();
10     static synchronized native public void moveZDown();
11     static synchronized native public void stopZ();
12     static synchronized native public int getZPos();
13
14     static synchronized native public void moveYInside();
15     static synchronized native public void moveYOutside();
16     static synchronized native public void stopY();
17     static synchronized native public int getYPos();
18
19     static synchronized native public int getSwitch1();
20     static synchronized native public int getSwitch2();
21     static synchronized native public int getSwitch1_2();
22
23     static synchronized native public void ledOn(int led);
24     static synchronized native public void ledsOff();
25 }
26
```

CREATION OF THE JNI INTERFACE



Generate Heading File

- ❑ Launch a console cmd.exe
- ❑ Considering the working directory (C:\STR\Labwork2), execute the command:

C:\STR\Labwork2\labwork2\src

- ❑ Then execute:

```
javac -h . Storage.java
dir
del *.class
```

- ❑ As you can see, the file **Storage.h** has been created, which needs to be included into the Visual Studio Project.

```
Command Prompt
Microsoft Windows [Version 10.0.22621.2428]
(c) Microsoft Corporation. All rights reserved.

C:\Users\faf>cd C:\STR\Labwork2\labwork2\src

C:\STR\Labwork2\labwork2\src>javac -h . Storage.java

C:\STR\Labwork2\labwork2\src>dir
Volume in drive C is Windows-SSD
Volume Serial Number is F6E8-A571

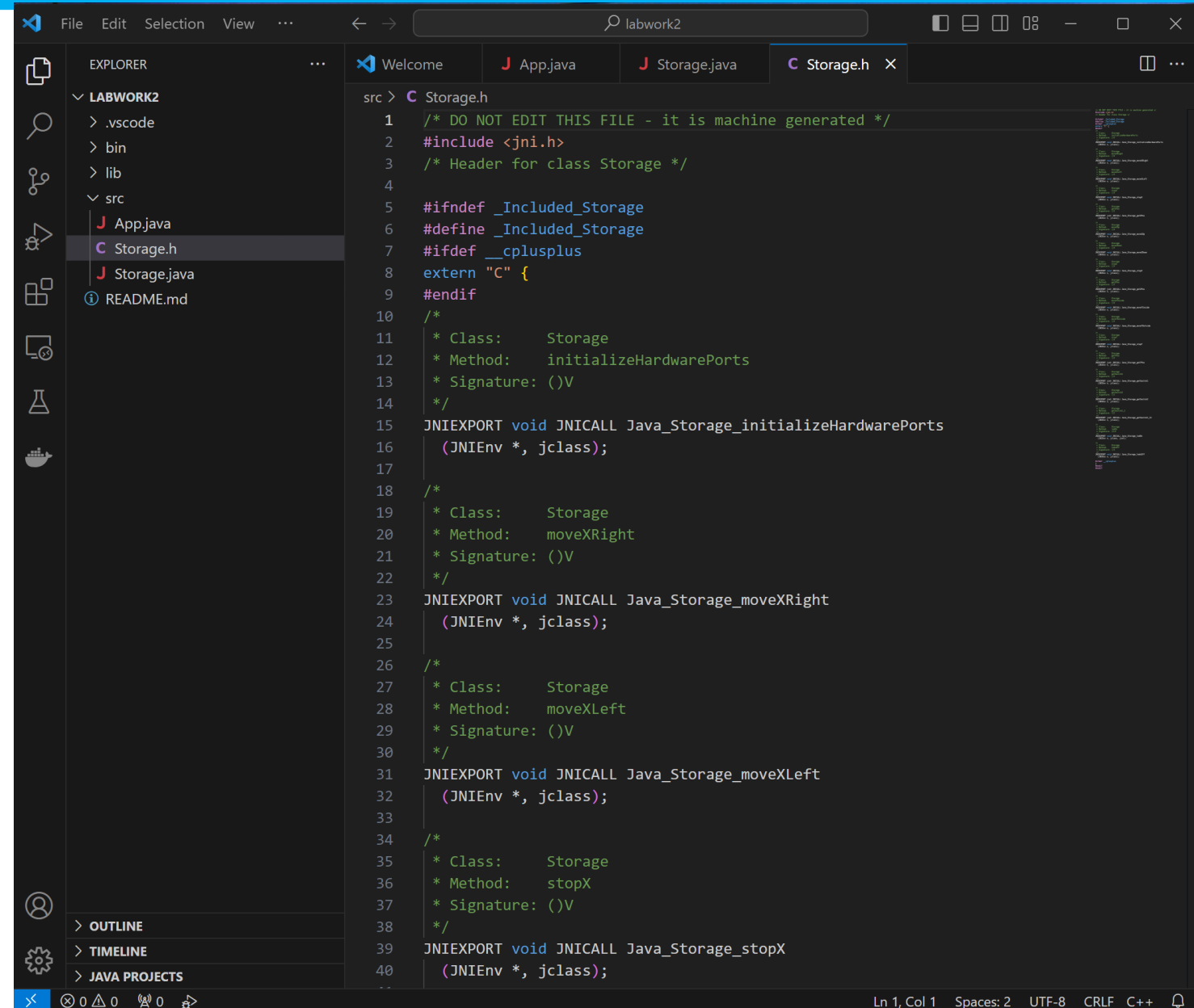
Directory of C:\STR\Labwork2\labwork2\src

2023-10-20  18:46    <DIR>          .
2023-10-20  18:22    <DIR>          ..
2023-09-29  20:08                134 App.java
2023-10-20  18:46                560 Storage.class
2023-10-20  18:46            2,912 Storage.h
2023-10-20  18:40            1,063 Storage.java
                4 File(s)              4,669 bytes
                2 Dir(s)  577,458,544,640 bytes free

C:\STR\Labwork2\labwork2\src>
```

CREATION OF THE JNI INTERFACE

The **obtained** signatures



```
1  /* DO NOT EDIT THIS FILE - it is machine generated */
2  #include <jni.h>
3  /* Header for class Storage */
4
5  #ifndef _Included_Storage
6  #define _Included_Storage
7  #ifdef __cplusplus
8  extern "C" {
9  #endif
10 /*
11  * Class:      Storage
12  * Method:     initializeHardwarePorts
13  * Signature:  ()V
14  */
15 JNIEXPORT void JNICALL Java_Storage_initializeHardwarePorts
16   (JNIEnv *, jclass);
17
18 /*
19  * Class:      Storage
20  * Method:     moveXRight
21  * Signature:  ()V
22  */
23 JNIEXPORT void JNICALL Java_Storage_moveXRight
24   (JNIEnv *, jclass);
25
26 /*
27  * Class:      Storage
28  * Method:     moveXLeft
29  * Signature:  ()V
30  */
31 JNIEXPORT void JNICALL Java_Storage_moveXLeft
32   (JNIEnv *, jclass);
33
34 /*
35  * Class:      Storage
36  * Method:     stopX
37  * Signature:  ()V
38  */
39 JNIEXPORT void JNICALL Java_Storage_stopX
40   (JNIEnv *, jclass);
```

CREATION OF THE JNI INTERFACE



In Visual Studio Project **storage** add the Java include path into the Visual Studio project properties.

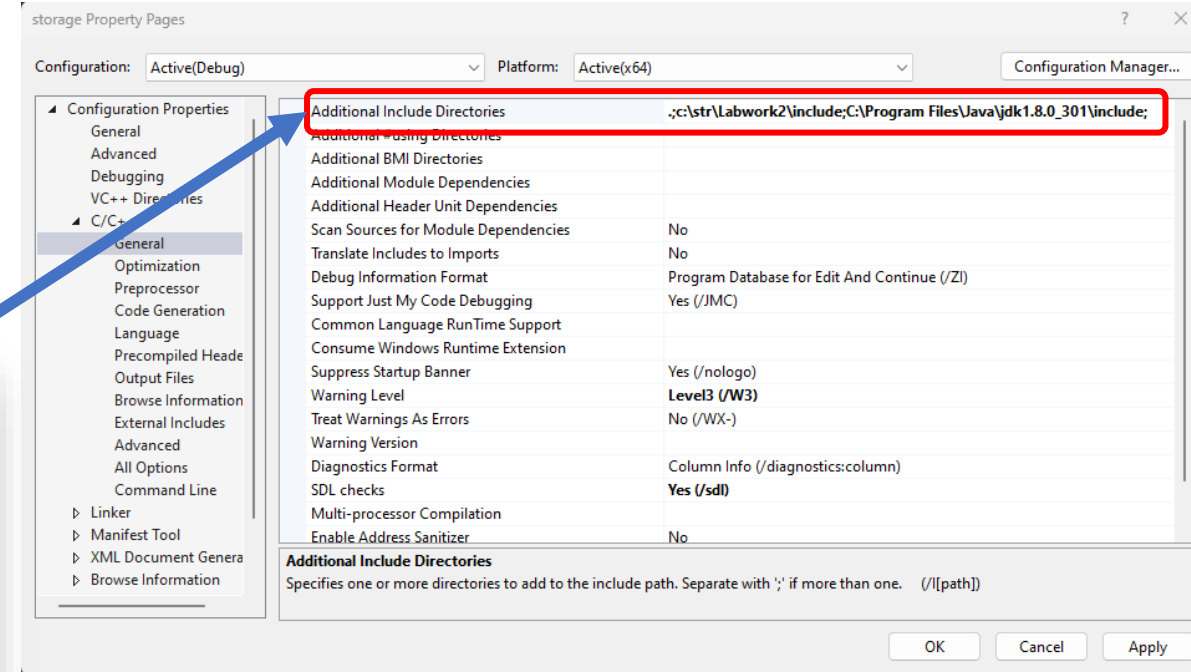
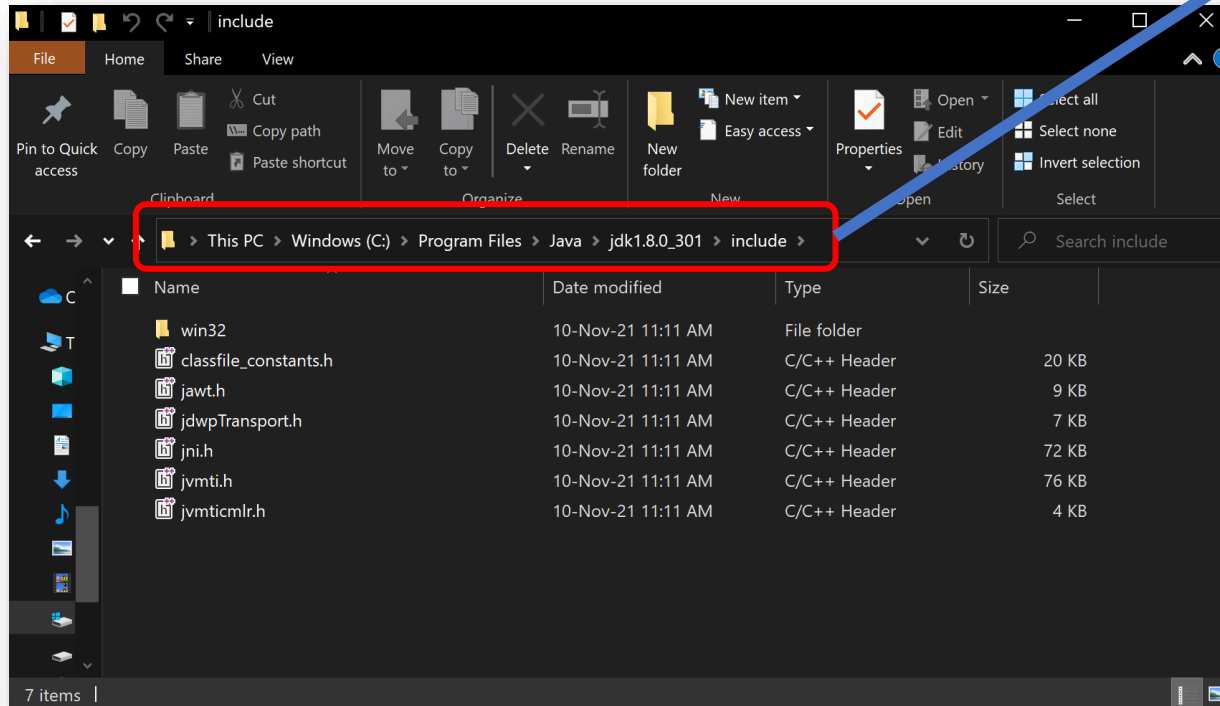


Must be something like this e.g. (depends on java version and installed path:

`C:\Program Files\Java\jdk-XXX\include;`

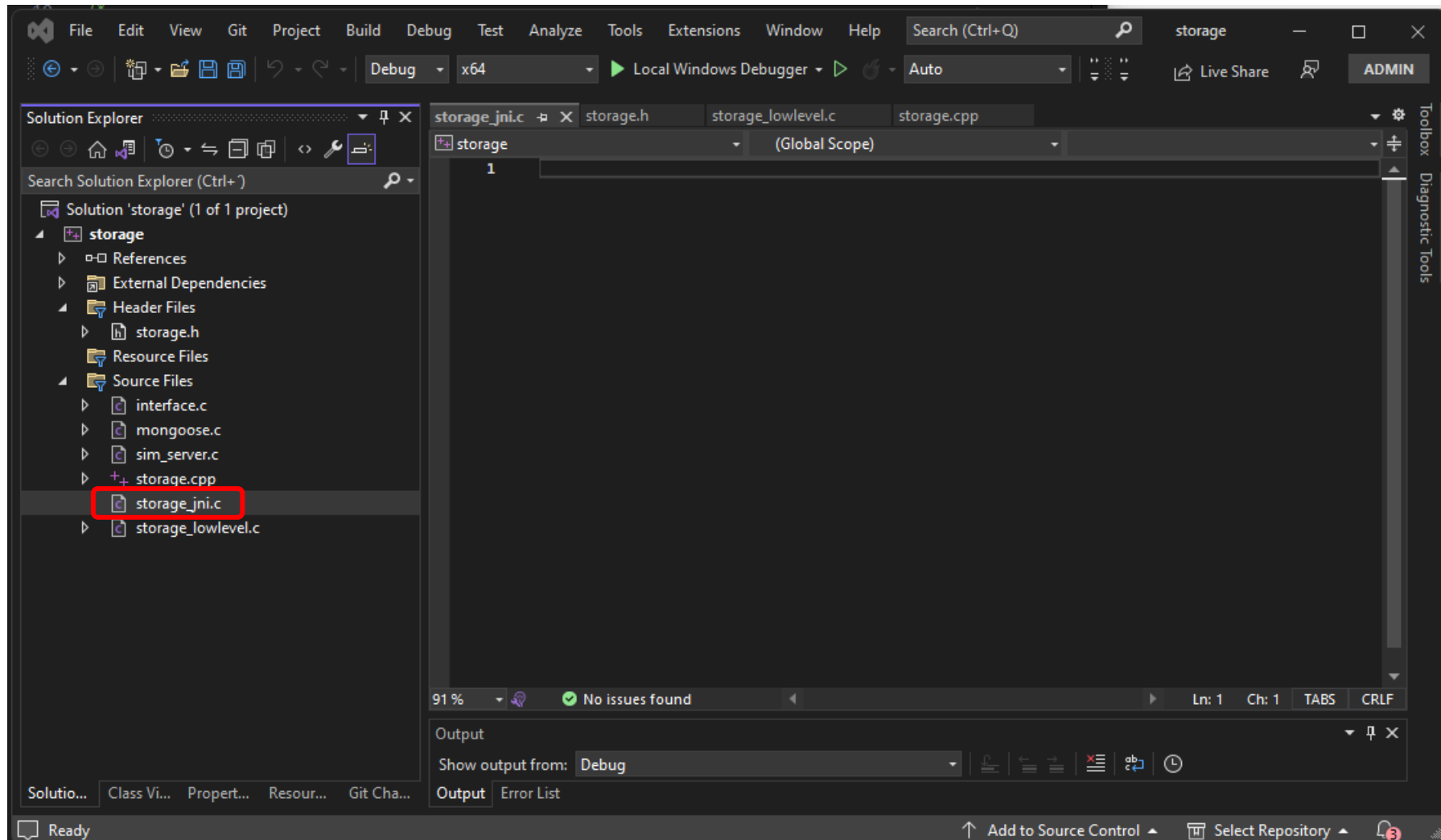
`C:\Program Files\Java\jdk-XXX\include \win32`

Split each path with a semi-colon (;)



CREATION OF THE JNI INTERFACE

Add new file `storage_jni.c` into your VS project



CREATION OF THE JNI INTERFACE



Copy the generated low-level functions from **Storage.h**

Complete as illustrated in the picture for all the functions

Functions called from the JNI stubs are specified in **storage.h** and developed in **storage_lowlevel.c**

The screenshot shows the Visual Studio IDE. On the left, the Solution Explorer displays the 'storage' project with the following structure:

- References
- External Dependencies
- Header Files
- Resource Files
- Source Files
 - interface.c
 - mongoose.c
 - sim_server.c
 - storage.cpp
 - storage_jni.c
 - storage_lowlevel.c

The main editor window shows the **storage_jni.c** file. The code defines several JNIEXPORT functions. The first function, **Java_Storage_initializeHardwarePorts**, is highlighted with a red box around its signature and implementation:

```
1 #include <jni.h>
2 #include <storage.h>
3
4
5 JNIEXPORT void JNICALL Java_Storage_initializeHardwarePorts(JNIEnv* jniEnv, jclass jclass) {
6     initializeHardwarePorts();
7 }
8
9 JNIEXPORT void JNICALL Java_Storage_moveXRight(JNIEnv* jniEnv, jclass jclass) {
10     moveXRight();
11 }
12
13 JNIEXPORT void JNICALL Java_Storage_moveXLeft(JNIEnv* jniEnv, jclass jclass) {
14     moveXLeft();
15 }
16
17 JNIEXPORT void JNICALL Java_Storage_stopX(JNIEnv* jniEnv, jclass jclass) {
18     stopX();
19 }
20
21 JNIEXPORT jint JNICALL Java_Storage_getXPos(JNIEnv* jniEnv, jclass jclass) {
22     return getXPos();
23 }
24
25 JNIEXPORT void JNICALL Java_Storage_moveZUp(JNIEnv* jniEnv, jclass jclass) {
26     moveZUp();
27 }
28
```

Attention to function ledOn!!!



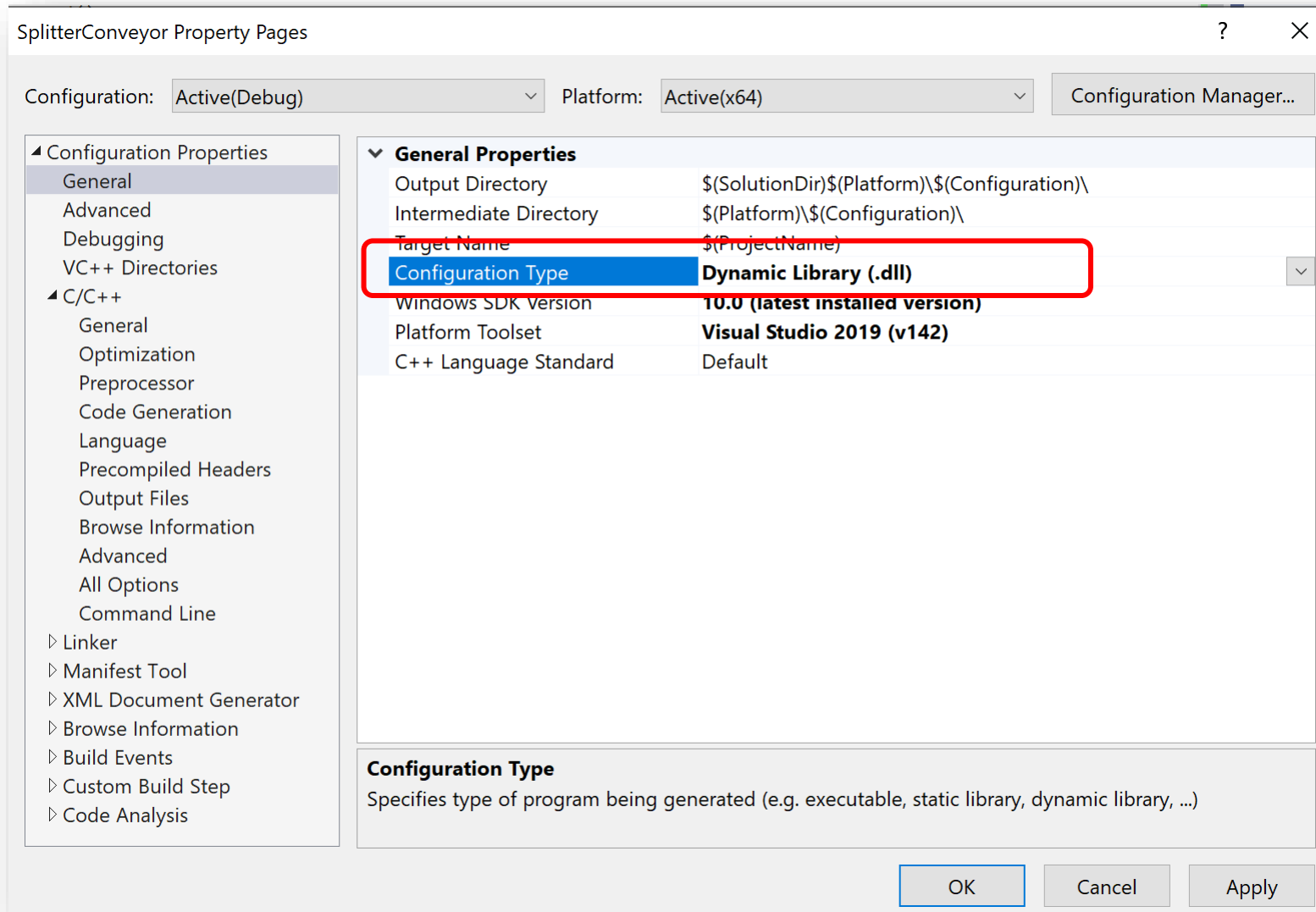
A close-up of the **Java_Storage_ledOn** function signature in **storage_jni.c**. The signature is highlighted with a red box:

```
JNIEXPORT void JNICALL Java_Storage_ledOn(JNIEnv* jniEnv, jclass jclass, jint n) {
    ledOn(n);
}
```

CREATION OF THE JNI INTERFACE



Transform Visual Studio project into **DLL**



Creation of the JNI interface

Build the project



Build the project and...



Hint: **don't try run it** (not an executable *.exe anymore)

```
Output
Show output from: Build
1>storage.cpp
1>LINK : C:\STR\Labwork2\storage\x64\Debug\storage.dll not found or not built by the last incremental link; performing full link
1> Creating library C:\STR\Labwork2\storage\x64\Debug\storage.lib and object C:\STR\Labwork2\storage\x64\Debug\storage.exp
1>storage.vcxproj -> C:\STR\Labwork2\storage\x64\Debug\storage.dll
1>Done building project "storage.vcxproj".
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Elapsed 00:02.852 =====
```

After that, copy the path for the generated DLL, as shown in the figure.

CREATION OF THE JNI INTERFACE

Code to load DLL from Java

Copy-past the DLL path inside `Storage.java` class



```
2
3 public class Storage {
4
5     static{
6         System.load(filename:"C:\\STR\\Labwork2\\storage\\x64\\Debug\\storage.dll
7     }
8
9     static synchronized native public void initializeHardwarePorts();
10    static synchronized native public void moveXRight();
11    static synchronized native public void moveXLeft();
12    static synchronized native public void stopX();
13    static synchronized native public int getXPos();
14
15    static synchronized native public void moveZUp();
16    static synchronized native public void moveZDown();
17    static synchronized native public void stopZ();
18    static synchronized native public int getZPos();
19
20    static synchronized native public void moveYInside();
21    static synchronized native public void moveYOutside();
22    static synchronized native public void stopY();
23    static synchronized native public int getYPos();
24
25    static synchronized native public int getSwitch1();
26    static synchronized native public int getSwitch2();
27    static synchronized native public int getSwitch1_2();
28
29    static synchronized native public void ledOn(int led);
30    static synchronized native public void ledsOff();
31 }
32
```



Hint: Write the empty string ("") first, then copy-past. This way, double backslashes are added automatically.

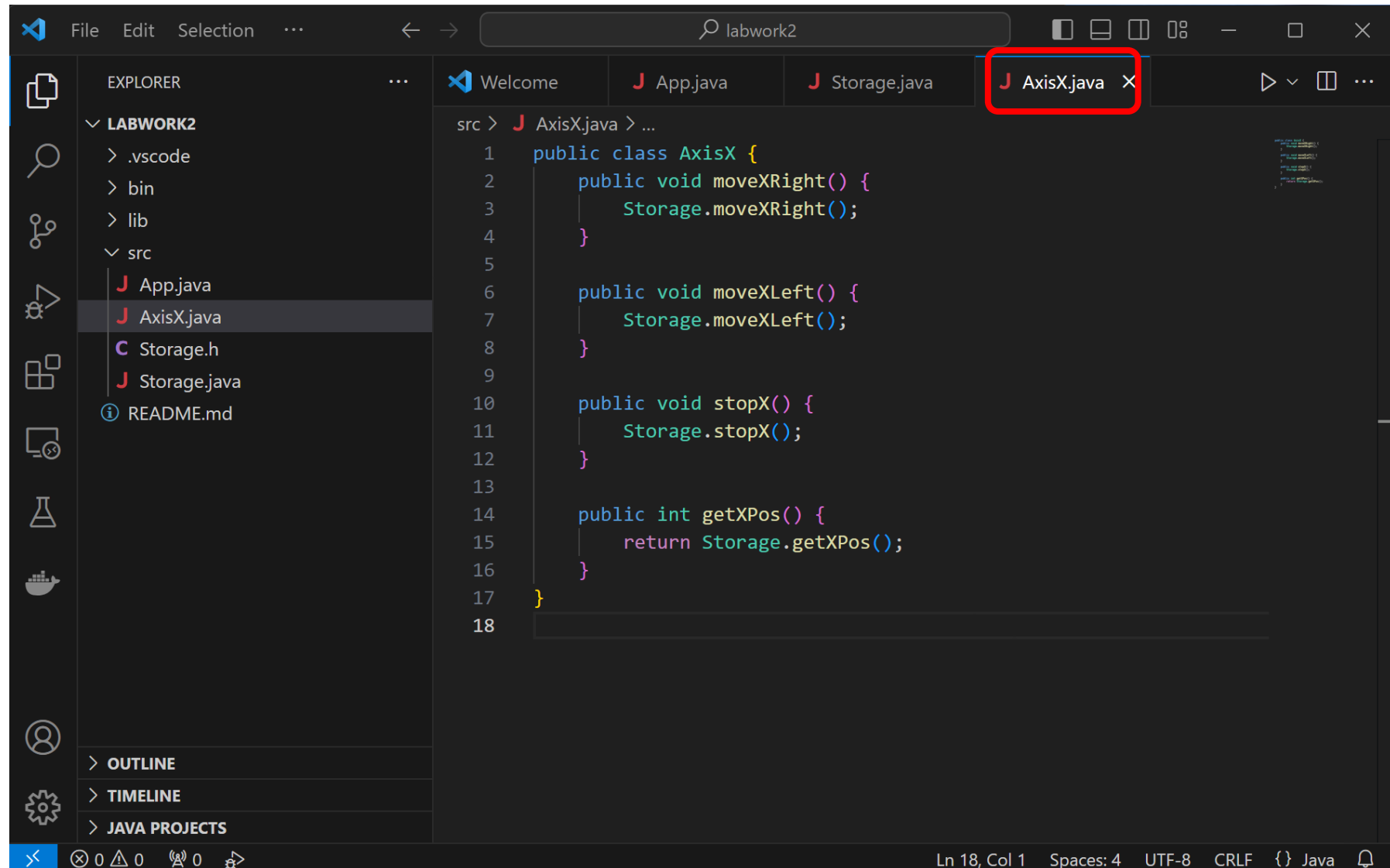
ACCESSING LOW-LEVEL FUNCTIONALITY



Add new Java class (1): Add a new class named **AxisX.java**



Copy the methods
illustrated in the
figure

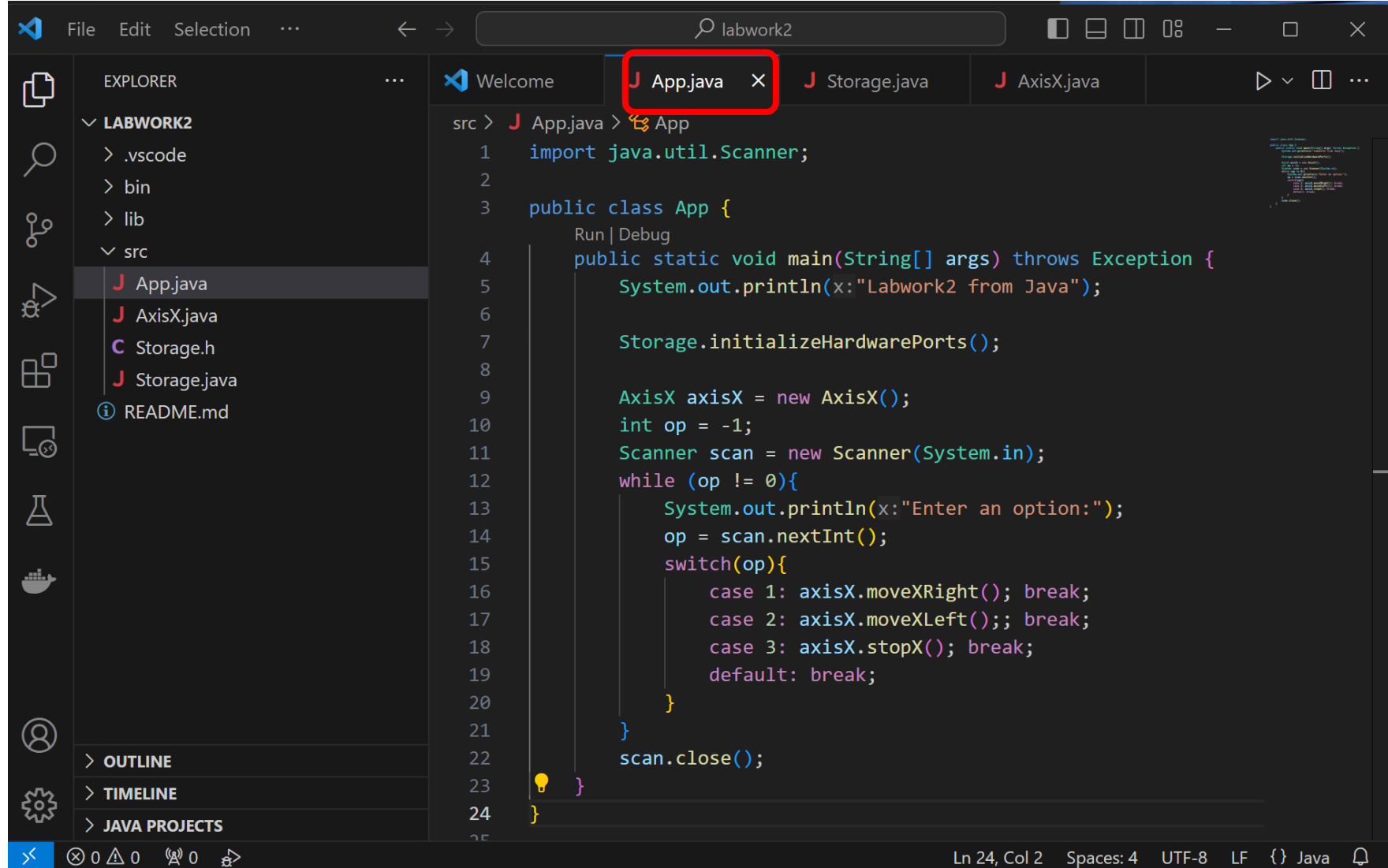


The screenshot shows the Visual Studio Code interface. In the Explorer sidebar on the left, the file **AxisX.java** is highlighted under the **src** folder. The main editor window displays the code for **AxisX.java**, which is a public class containing four methods: `moveXRight()`, `moveXLeft()`, `stopX()`, and `getXPos()`. Each method delegates the call to the corresponding method in the `Storage` class. The `AxisX.java tab in the top editor bar is highlighted with a red rectangle. The status bar at the bottom indicates the current position is Line 18, Column 1.`

```
src > J AxisX.java > ...
1  public class AxisX {
2      public void moveXRight() {
3          Storage.moveXRight();
4      }
5
6      public void moveXLeft() {
7          Storage.moveXLeft();
8      }
9
10     public void stopX() {
11         Storage.stopX();
12     }
13
14     public int getXPos() {
15         return Storage.getXPos();
16     }
17 }
18
```

Testing from Java Application

Open `App.java` class



```
File Edit Selection ... labwork2
EXPLORER
  LABWORK2
    .vscode
    bin
    lib
    src
      App.java
      AxisX.java
      Storage.h
      Storage.java
      README.md
  OUTLINE
  TIMELINE
  JAVA PROJECTS

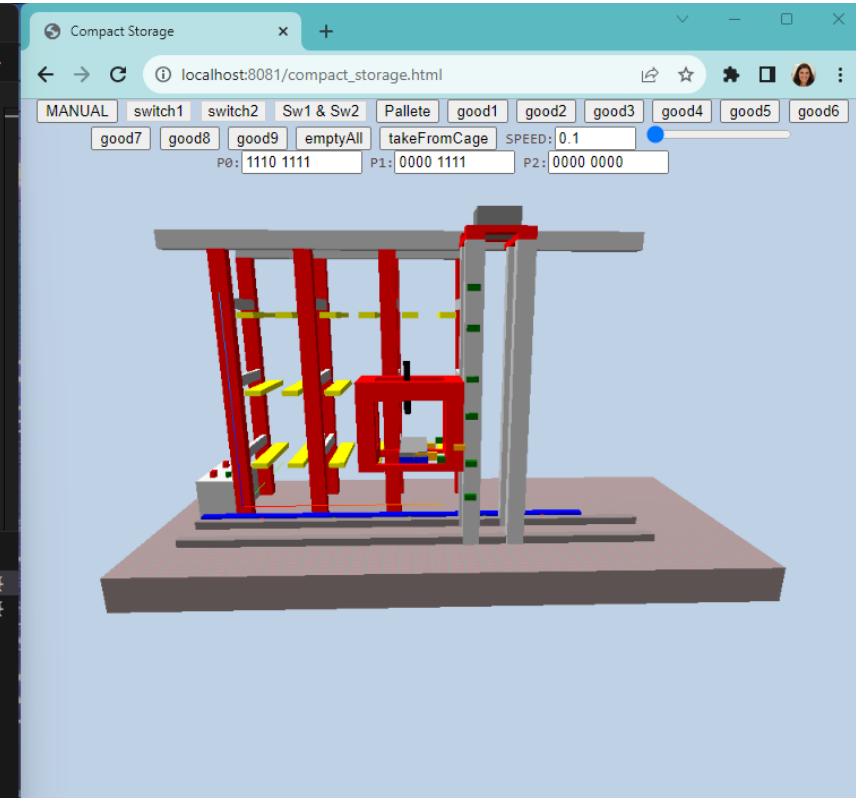
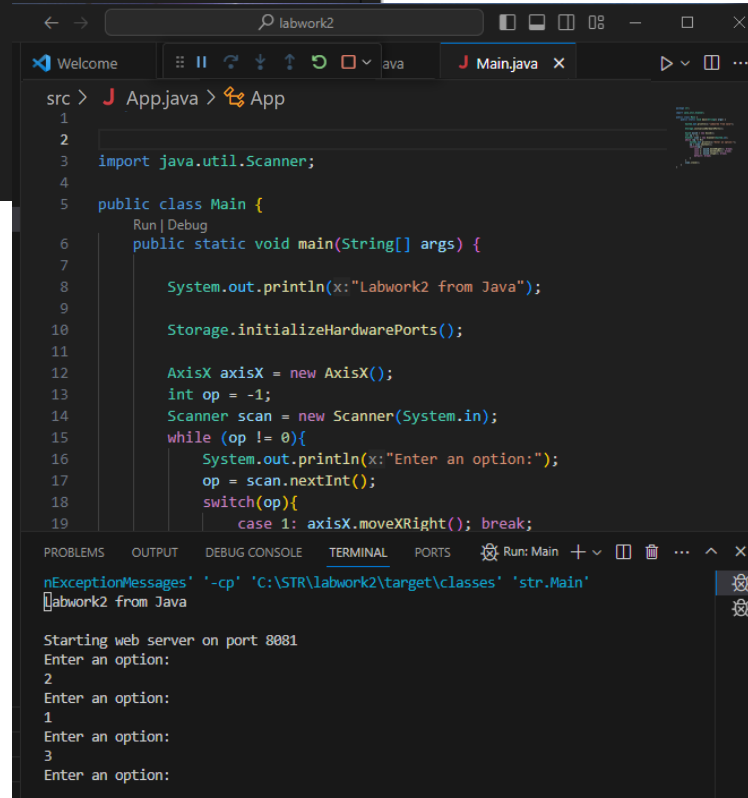
src > J App.java > App
1  import java.util.Scanner;
2
3  public class App {
4      Run | Debug
5      public static void main(String[] args) throws Exception {
6          System.out.println(x:"Labwork2 from Java");
7
8          Storage.initializeHardwarePorts();
9
10         AxisX axisX = new AxisX();
11         int op = -1;
12         Scanner scan = new Scanner(System.in);
13         while (op != 0){
14             System.out.println(x:"Enter an option:");
15             op = scan.nextInt();
16             switch(op){
17                 case 1: axisX.moveXRight(); break;
18                 case 2: axisX.moveXLeft(); break;
19                 case 3: axisX.stopX(); break;
20                 default: break;
21             }
22         }
23         scan.close();
24     }
25 }
```

NOVA

The screenshot shows an IDE window with the following content:

- Top bar: Search icon and text "labwork2".
- Tab bar: "Welcome", "Storage.java", "AxisX.java", "Main.java X".
- Toolbar: Run button (play icon) is highlighted with a red box.
- Editor:


```
src > J App.java > App
1
2
3 import java.util.Scanner;
4
5 public class Main {
6     public static void main(String[] args) {
7
8         System.out.println("Labwork2 from Java");
9
10        Storage.initializeHardwarePorts();
11
12        AxisX axisX = new AxisX();
```
- Bottom panel: Preview of the code, showing lines 1-4.



TESTING FROM JAVA APPLICATION

If you get unsatisfied link error



Well, that's bad luck...

We need to repeat the process of using the [Storage](#) through the JNI interface. Repeat the steps of this process very carefully.



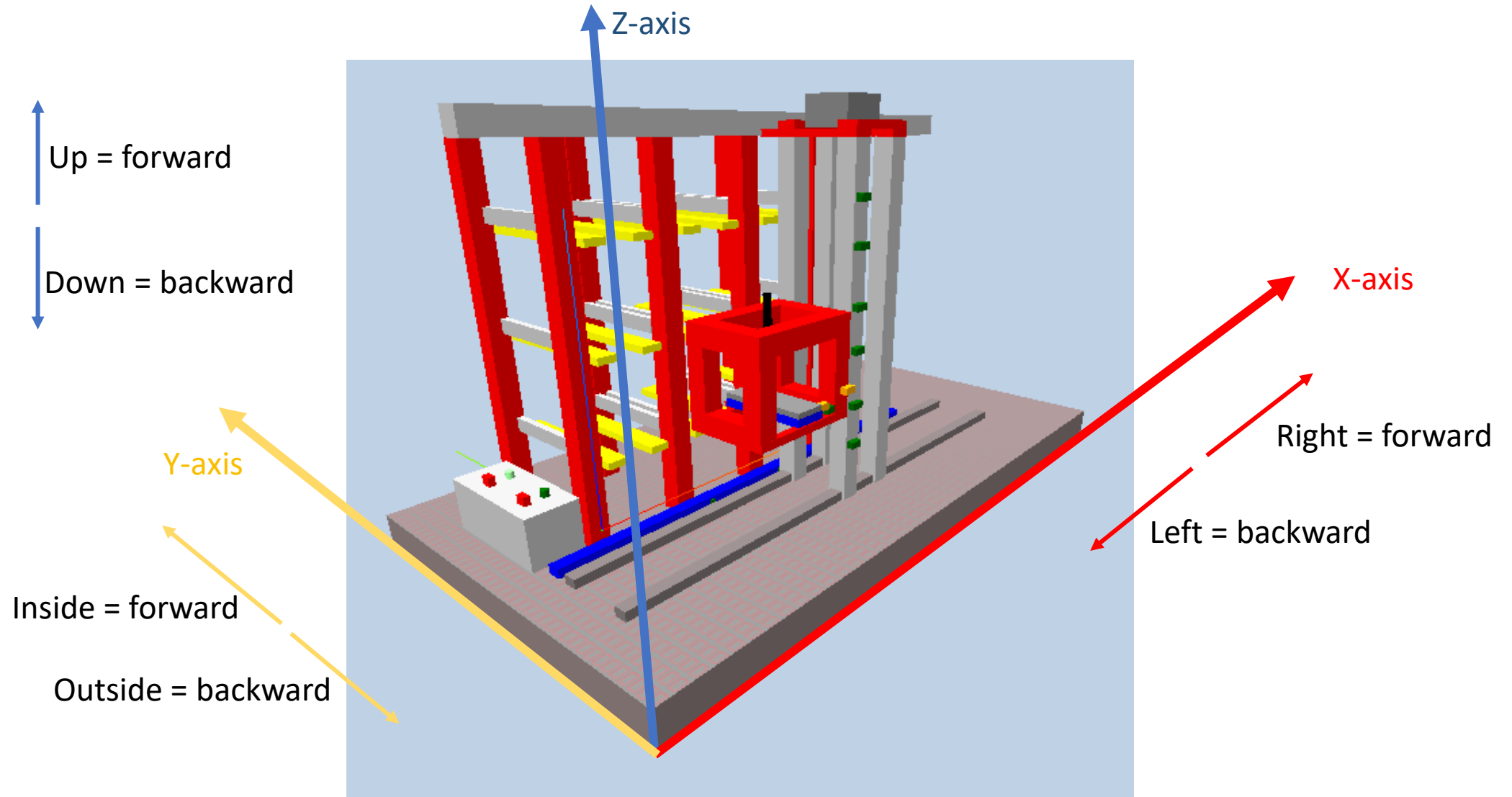
```
Output - labwork2 (run) x
Exception in thread "main" java.lang.UnsatisfiedLinkError:
    at
    at
    at
C:\Users\ai0\AppData\Local\NetBeans\Cache\12.5\executor-snippets\run.xml:111: The following error occurred while executing this line:
C:\Users\ai0\AppData\Local\NetBeans\Cache\12.5\executor-snippets\run.xml:94: Java returned: 1
BUILD FAILED (total time: 11 seconds)
```

PART 2

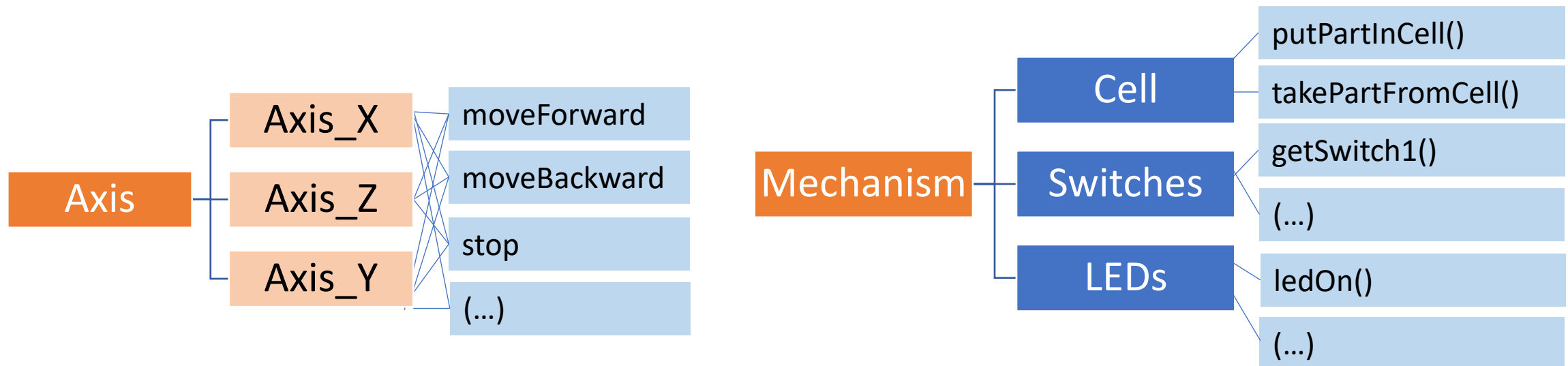
Java Real-Time



LABWORK 2 – AXIS ABSTRACTION



- ❑ Having the class **Storage.java** already developed (the DLL), at the Java side we need to organize our classes into a more logical and straightforward way.
- ❑ This should be done by aggregating the storage functionality as several well identified classes, some of them as Java threads.
- ❑ For instance, we can define concrete classes for axis operation and a class named mechanism, which aggregates the functionality for the cell movements, the switches, and the LEDs.



THE AXIS INTERFACE



Let's start by providing an interface definition for the Axis: Java interface -> [Axis.java](#)

The screenshot shows the Visual Studio Code editor with a project named 'labwork2'. The Explorer sidebar on the left shows the file structure: .vscode, bin, lib, and src. Under src, there are files App.java, Axis.java (selected), AxisX.java, Storage.h, and Storage.java, along with a README.md file. The main editor window shows the code for 'Axis.java'. The code defines a public interface 'Axis' with four methods: moveForward(), moveBackward(), stop(), and getPos(). It also includes a comment and a method signature for 'gotoPos(int pos)'. The word 'interface' and the name 'Axis' are highlighted with red boxes.

```
1  
2 public interface Axis {  
3     public void moveForward();  
4     public void moveBackward();  
5     public void stop();  
6     public int getPos();  
7  
8     // new method to implement in JAVA  
9     public void gotoPos(int pos);  
10  
11 }  
12
```

An **interface** is a completely "**abstract class**" that is used to group related methods with empty bodies

https://www.w3schools.com/java/java_interface.asp

A class that implements a specific interface must implement the methods of the interface.

The interface has several methods that are used in each axis.

Although each axis has distinct implementation, in an abstract way they behave similarly.....

As such, interface **Axis** specifies and describes the behavior of each axis in an abstract way.

THE “NEW” AXISX CLASS



```
src > J AxisX.java > ...
1  public class AxisX implements Axis{
2      @Override
3      public void moveForward() {
4          Storage.moveXRight();
5      }
6
7      @Override
8      public void moveBackward() {
9          Storage.moveXLeft();
10     }
11
12     @Override
13     public void stop() {
14         Storage.stopX();
15     }
16
17     @Override
18     public int getPos() {
19         return Storage.getXPos();
20     }
21
22     @Override
23     public void gotoPos(int pos) {
24         // TODO Auto-generated method stub
25         throw new UnsupportedOperationException(message:"Unimplemented method 'gotoPos'");
26
27         //to be developed in JAVA
28     }
29 }
30
```

Each concrete **axis** can follow the **Axis interface** and implement the concrete functionality of each Axis's methods.

When a class implements an axis, it must **implement all** the interface's methods.

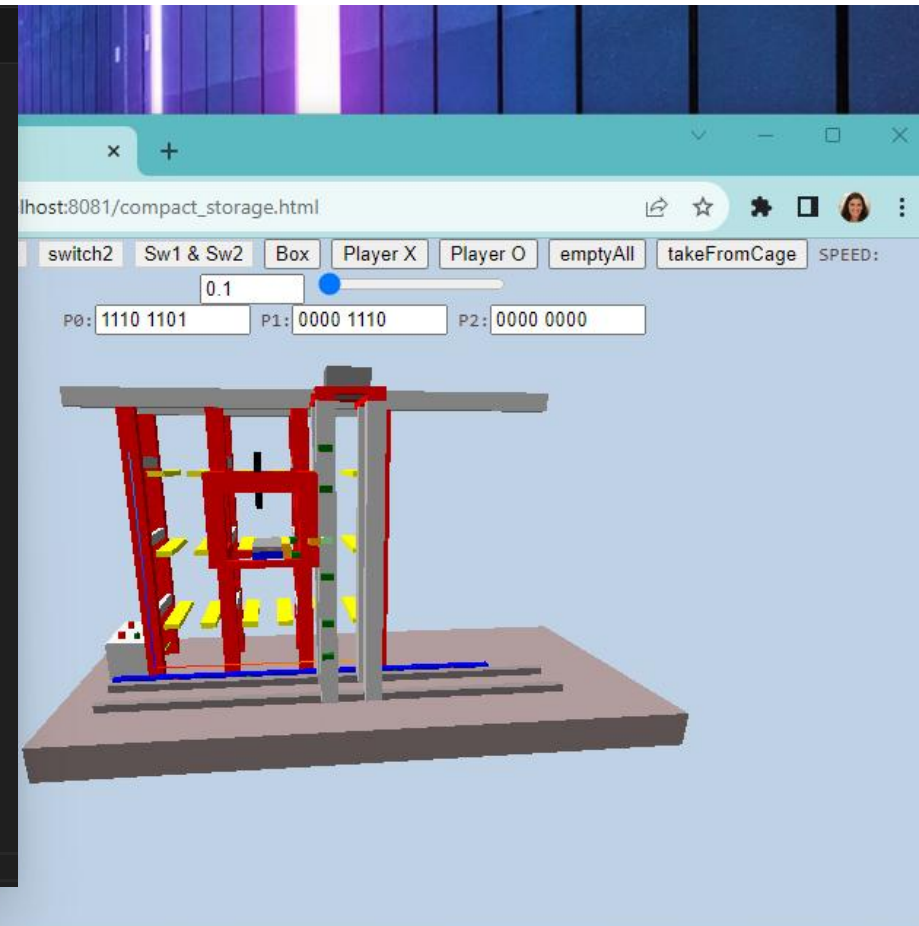
Based on the given interface, try develop **three classes** for each **axis** existing in the physical system.

Testing from Java Application

App.java class -> Change it accordingly with the new Axis methods and test



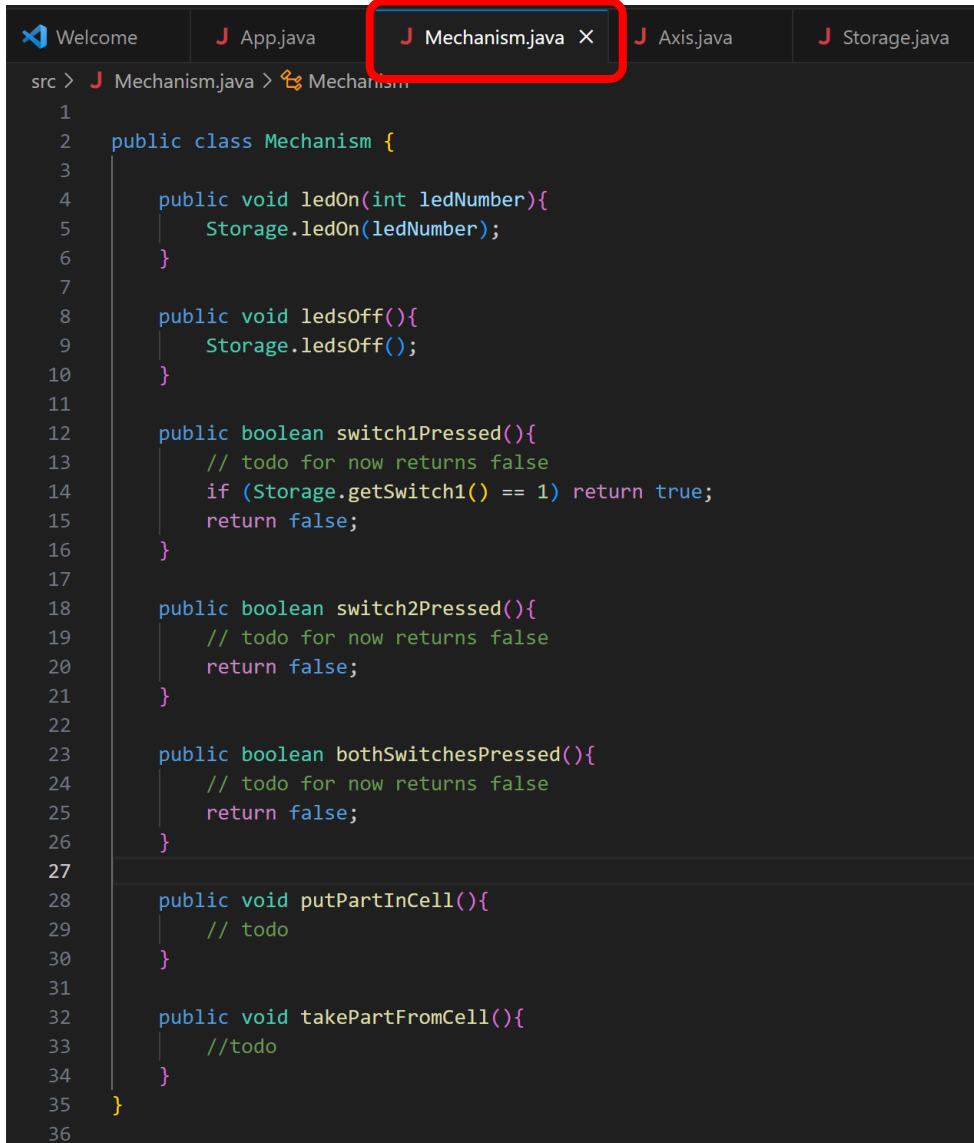
```
src > J App.java > ...
1  import java.util.Scanner;
2
3  public class App {
4      Run | Debug
5      public static void main(String[] args) throws Exception {
6          System.out.println(x:"Labwork2 from Java");
7
8          Storage.initializeHardwarePorts();
9
10         AxisX axisX = new AxisX();
11         int op = -1;
12         Scanner scan = new Scanner(System.in);
13         while (op != 0){
14             System.out.println(x:"Enter an option:");
15             op = scan.nextInt();
16             switch(op){
17                 case 1: axisX.moveForward(); break;
18                 case 2: axisX.moveBackward(); break;
19                 case 3: axisX.stop(); break;
20                 default: break;
21             }
22         }
23         scan.close();
24     }
25 }
```



THE CLASS MECHANISM



Create a new class **Mechanism.java**



```
1
2 public class Mechanism {
3
4     public void ledOn(int ledNumber){
5         Storage.ledOn(ledNumber);
6     }
7
8     public void ledsOff(){
9         Storage.ledsOff();
10    }
11
12    public boolean switch1Pressed(){
13        // todo for now returns false
14        if (Storage.getSwitch1() == 1) return true;
15        return false;
16    }
17
18    public boolean switch2Pressed(){
19        // todo for now returns false
20        return false;
21    }
22
23    public boolean bothSwitchesPressed(){
24        // todo for now returns false
25        return false;
26    }
27
28    public void putPartInCell(){
29        // todo
30    }
31
32    public void takePartFromCell(){
33        //todo
34    }
35 }
36
```

The class mechanism deals with:

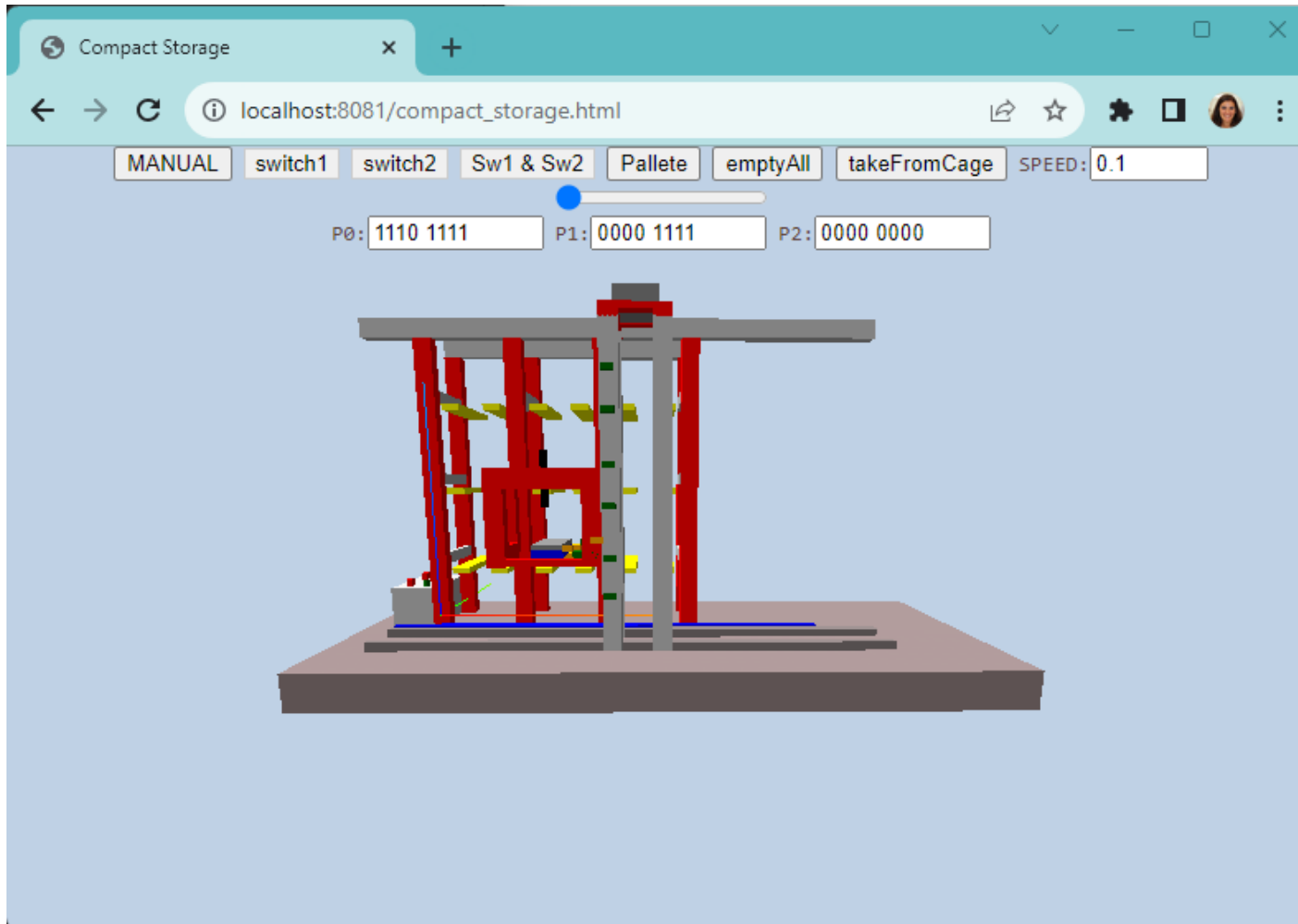
- the LEDs
- the switches
- the movements to insert and take parts from cell

Remember, some of these methods might be static, others not

Static methods do not use any instance variables of any object of the class they are defined in. Static methods take all the data from parameters and compute something from those parameters, with no reference to variables.

<https://www.tutorialspoint.com/Java-static-method>

IDENTIFICATION OF NEEDED JAVA THREADS



- ☐ Start thinking of which THREADS are needed to be executed synchronously or in parallel.
- ☐ Tip: start by implementing **thread_calibrate**, **thread_gotoX** and **thread_gotoZ...**
- ☐ Draw a first diagram illustrating the threads and the interactions between them.

TO STUDY....



Check....

<https://docs.oracle.com/javase/8/docs/api/index.html>

PREV CLASS NEXT CLASS FRAMES NO FRAMES
SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

compact1, compact2, compact3
java.lang.
Class Thread
java.lang.Object
java.lang.Thread

All Implemented Interfaces:
Runnable

Direct Known Subclasses:
ForkJoinWorkerThread

public class Thread
extends Object
implements Runnable

A *thread* is a thread of execution in a program. The Java Virtual Machine allows an application to have multiple threads of execution running concurrently.

Every thread has a priority. Threads with higher priority are executed in preference to threads with lower priority. Each thread may or may not also be marked as a daemon. When code running in some thread creates a new Thread object, the new thread has its priority initially set equal to the priority of the creating thread, and is a daemon thread if and only if the creating thread is a daemon.

When a Java Virtual Machine starts up, there is usually a single non-daemon thread (which typically calls the method name of some designated class). The Java Virtual Machine continues to execute threads until either of the following occurs:

- The exit method of class Runtime has been called and the security manager has permitted the exit operation to take place.
- All threads that are not daemon threads have died, either by returning from the call to the run method or by throwing an exception that propagates beyond the run method.

There are two ways to create a new thread of execution. One is to declare a class to be a subclass of Thread. This subclass must override the run method of class Thread. An instance of the subclass can then be allocated and started. For example, a thread that computes primes larger than a stated value could be written as follows:

```
class PrimeThread extends Thread {  
    long minPrime;  
    PrimeThread(long minPrime) {  
        this.minPrime = minPrime;  
    }  
}
```

OVERVIEW PACKAGE CLASS USE TREE DEPRECATED INDEX HELP
PREV PACKAGE NEXT PACKAGE FRAMES NO FRAMES

Package java.util.concurrent
Utility classes commonly useful in concurrent programming.
See: Description

Interface Summary

Interface	Description
BlockingDeque<E>	A Deque that additionally supports blocking operations that wait for the deque to become non-empty when retrieving an element, and wait for space to become available in the deque when storing an element.
BlockingQueue<E>	A Queue that additionally supports operations that wait for the queue to become non-empty when retrieving an element, and wait for space to become available in the queue when storing an element.
Callable<V>	A task that returns a result and may throw an exception.
CompletableFuture.AsyncronousCompletionTask	A marker interface identifying asynchronous tasks produced by async methods.
CompletionService<V>	A service that decouples the production of new asynchronous tasks from the consumption of the results of completed tasks.
CompletionStage<T>	A stage of a possibly asynchronous computation, that performs an action or computes a value when another CompletionStage completes.
ConcurrentMap<K,V>	A Map providing thread safety and atomicity guarantees.
ConcurrentNavigableMap<K,V>	A ConcurrentMap supporting NavigableMap operations, and recursively so for its navigable sub-maps.
Delayed	A mix-in style interface for marking objects that should be acted upon after a given delay.
Executor	An object that executes submitted Runnable tasks.
ExecutorService	An Executor that provides methods to manage termination and methods that can produce a Future for tracking progress of one or more asynchronous tasks.
ForkJoinPool.ForkJoinWorkerThreadFactory	Factory for creating new ForkJoinWorkerThreads .
ForkJoinPool.ManagedBlocker	Interface for extending managed parallelism for tasks running in ForkJoinPools .

<https://docs.oracle.com/javase/8/docs/api/index.html?java/util/concurrent/package-summary.html>



KEEP UP THE
GOOD
WORK!